

12.3 软件组合：三叉组合树 (Trinary Combining Tree)

trinary-combining-tree & Trae

2026 年 2 月 9 日

摘要

本文档仿照《The Art of Multiprocessor Programming》第 12.3 节的风格与结构，详细阐述了新设计的**三叉组合树 (Trinary Combining Tree)**的数据结构、实现原理及运行过程。该结构是对经典二叉组合树的扩展，通过增加节点的度数 (Degree) 至 3，旨在降低树的高度并提高并发吞吐量。

目录

1	引言 (Introduction)	2
2	数据结构 (Data Structure)	2
2.1	节点状态 (Node Status)	2
2.2	字段定义 (Fields)	2
3	算法实现 (Algorithm Implementation)	3
3.1	第一阶段：预合并 (Pre-combining)	4
3.2	第二阶段：合并 (Combining)	4
3.3	第三阶段：操作 (Operation)	5
3.4	第四阶段：分发 (Distribution)	6
4	运行过程示例 (An Execution Trace)	6
5	分析 (Analysis)	7
5.1	正确性 (Correctness)	7
5.2	性能 (Performance)	8

1 引言 (Introduction)

在并发编程中，单一的共享计数器（如基于 CAS 的 `AtomicInteger`）往往成为性能瓶颈。当大量线程同时尝试更新同一内存地址时，会引发严重的内存争用（Memory Contention），导致内存总线或片上互连（On-chip Interconnect）拥塞，从而使性能急剧下降。

软件组合 (Software Combining) 技术通过将多个并发请求组织成树状结构来解决这一问题。线程从叶子节点进入，沿着路径向根节点移动。如果多个线程在同一个节点相遇，它们会将请求“合并”。其中一个线程（主动线程）继续向上携带合并后的请求，而其他线程（被动线程）则在此等待。当主动线程完成操作并返回时，它将结果分发给等待的被动线程。

本报告介绍的**三叉组合树**将每个节点的合并能力扩展为 3，即每个节点最多可以汇聚 3 个线程的请求。相比于二叉树的 $\log_2 N$ （其中 N 表示并发线程数）高度，三叉树的高度降低为 $\log_3 N$ ，理论上能减少延迟。

2 数据结构 (Data Structure)

三叉组合树的核心组件是 `TriNode` 类。与二叉树节点不同，三叉节点必须能够协调最多三个线程的交互。

2.1 节点状态 (Node Status)

节点的状态由枚举 `CStatus3` 定义，包含以下状态：

- **IDLE**: 节点处于空闲状态，无线程活动。
- **FIRST**: 第一个线程到达节点，暂时处于等待后续线程或准备向上移动的状态。
- **SECOND**: 第二个线程到达节点。
- **THIRD**: 第三个线程到达节点。
- **RESULT**: 操作已完成，结果已就绪，正在分发给被动线程。
- **ROOT**: 特殊状态，标识根节点。

2.2 字段定义 (Fields)

`TriNode` 包含以下关键字段用于同步和数据交换：

```
1 public class TriNode {  
2     enum CStatus3 {IDLE, FIRST, SECOND, THIRD, RESULT, ROOT};  
3 }
```

```

4  boolean locked[] = new boolean[2]; // 细粒度锁控制
5  int num_nodes; // 当前汇聚的被动线程数量
6  int ready; // 已准备好数据的被动线程计数
7  CStatus3 cStatus; // 当前组合状态
8
9  int firstValue; // FIRST线程的值
10 int secondValue[] = new int[2]; // SECOND和THIRD线程的值
11 int result[] = new int[2]; // 分发给SECOND和THIRD的结果
12 TriNode parent; // 父节点引用
13 // ...
14 }

```

- secondValue[] 和 result[] 数组的大小为 2，分别对应 SECOND 和 THIRD 线程的数据槽位。
- locked[] 数组用于在组合协议的不同阶段协调主动线程与被动线程的步调。

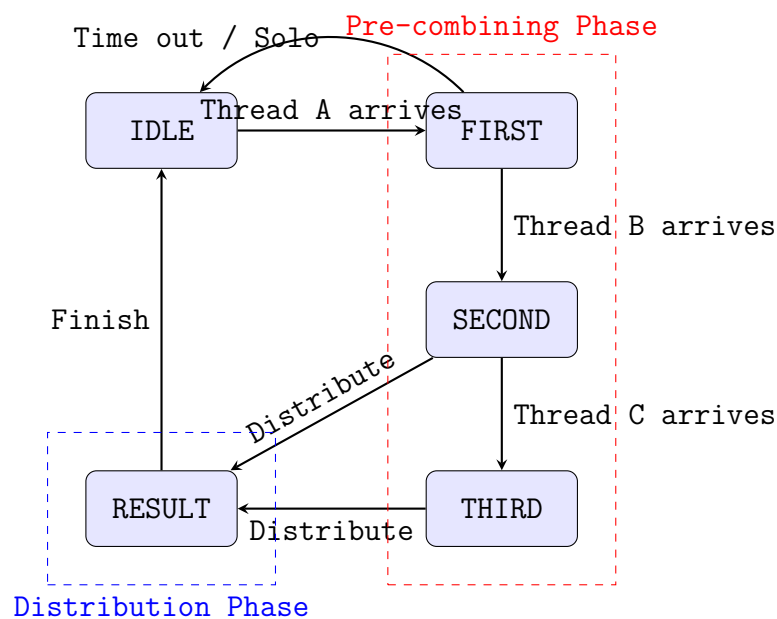


图 1: TriNode 状态转换示意图

3 算法实现 (Algorithm Implementation)

二叉组合树的协议分为四个阶段：预合并 (Pre-combining)、合并 (Combining)、操作 (Operation) 和分发 (Distribution)。入口方法为 `getAndIncrement()`。

3.1 第一阶段：预合并 (Pre-combining)

线程首先根据 ThreadID 映射到叶子节点，然后调用 precombine()。该方法决定了线程的角色 (FIRST, SECOND 或 THIRD)。

```

1 synchronized boolean precombine() throws InterruptedException {
2     while (locked[1] || num_nodes >= 2) wait(); // 等待上一轮清理或名额空缺
3     switch (cStatus) {
4         case IDLE:
5             cStatus = CStatus3.FIRST;
6             return true; // 成为主动线程
7         case FIRST:
8             locked[0] = true; // 锁定，阻止FIRST过早进行combine
9             num_nodes = 1;
10            cStatus = CStatus3.SECOND;
11            return false; // 成为被动线程
12        case SECOND:
13            num_nodes = 2;
14            cStatus = CStatus3.THIRD;
15            return false; // 成为被动线程
16        // ...
17    }
18 }

```

- **FIRST**: 返回 true，表示它将作为主动线程继续向上。
- **SECOND/THIRD**: 返回 false，它们将在后续的 op() 方法中等待。
- 注意 locked[0] = true: 这是 SECOND 线程设置的，用于通知 FIRST 线程“有同伴到了，请稍等我准备数据”。

3.2 第二阶段：合并 (Combining)

主动线程 (FIRST) 在向上移动前，调用 combine() 来收集被动线程的值。

```

1 synchronized int combine(int combined) throws InterruptedException {
2     // 等待被动线程准备好 (locked[0]被清除 且 ready数达标)
3     while (locked[0] || ready < num_nodes) wait();
4
5     locked[1] = true; // 锁定后续，防止新一轮干扰
6     firstValue = combined;
7
8     switch (cStatus) {
9         case FIRST: return firstValue; // 无人合并
10        case SECOND: return firstValue + secondValue[0]; // 合并1个

```

```

11         case THIRD: return firstValue + secondValue[0] + secondValue[1]; //
           合并2个
12         // ...
13     }
14 }

```

主动线程在此处计算局部子树的总和 (Prefix Sum), 准备携带该总和向父节点进发。

3.3 第三阶段：操作 (Operation)

这是协议的关键交互点。

- **主动线程**: 到达树的顶端 (或某个中间节点停止点) 执行实际的加法操作, 或者从父节点获取结果。
 - “到达树的顶端 (或某个中间节点停止点)” : 对应 TriTree.java 中 getAndIncrement 方法的第一阶段循环。如果 precombine() 返回 true (状态 IDLE → FIRST), 线程继续向上; 如果返回 false (遇到非空闲节点或根节点), 线程停止上升, 该节点即为停止点 stop。
 - “执行实际的加法操作, 或者从父节点获取结果” : 对应 TriNode.java 中 op() 方法的分支逻辑。如果停止点是 ROOT, 线程执行全局加法 (result += combined); 如果停止点是中间节点 (此时线程在该节点角色为 SECOND 或 THIRD), 它进入等待状态 (wait()), 直到父节点的主动线程完成操作并分发结果。
- **被动线程**: 调用 op() 实际上是进入等待状态, 并向主动线程提交数据。

被动线程在 op() 中的逻辑:

```

1 // 在 TriNode.java 的 op() 方法中
2 case SECOND:
3 case THIRD:
4     int myIndex = ready;
5     secondValue[myIndex] = combined; // 提交自己的值
6     ready++;
7     if (ready >= num_nodes) {
8         locked[0] = false; // 所有被动线程就绪, 释放锁, 允许FIRST继续
9         notifyAll();
10    }
11    while (cStatus != CStatus3.RESULT) wait(); // 等待结果
12    // ... 获取结果并返回

```

3.4 第四阶段：分发 (Distribution)

当主动线程拿到最终结果 (prior) 后，它在回程路上调用 `distribute()` 将结果分发给被动线程。

```

1 synchronized void distribute(int prior) throws InterruptedException {
2     switch (cStatus) {
3         case SECOND:
4             result[0] = prior + firstValue; // 分发给SECOND
5             cStatus = CStatus3.RESULT;
6             break;
7         case THIRD:
8             result[0] = prior + firstValue; // 分发给SECOND
9             result[1] = result[0] + secondValue[0]; // 分发给THIRD
10            cStatus = CStatus3.RESULT;
11            break;
12            // ...
13    }
14    notifyAll(); // 唤醒等待的被动线程
15 }

```

这里实现了结果的**序列化**。假设 FIRST 增加 v_1 , SECOND 增加 v_2 , THIRD 增加 v_3 。

- FIRST 得到 `prior`。
- SECOND 得到 `prior +  $v_1$` 。
- THIRD 得到 `prior +  $v_1 + v_2$` 。

这确保了所有线程虽然是并发执行，但获得的返回值是连续且唯一的，就像它们排队执行一样。

4 运行过程示例 (An Execution Trace)

假设有线程 A, B, C 同时到达同一个节点 N ，且都试图增加 1。

1. Arrival:

- A 到达，见状态 IDLE，变为 FIRST，返回 `true`。
- B 到达，见状态 FIRST，变为 SECOND，设置 `locked[0]=true`，返回 `false`。
- C 到达，见状态 SECOND，变为 THIRD，返回 `false`。

2. Coordination:

- A 调用 `combine()`，发现 `locked[0]` 为 `true`，等待。

- B 进入 `op()`, 写入 `secondValue[0]=1`, `ready` 变为 1。
- C 进入 `op()`, 写入 `secondValue[1]=1`, `ready` 变为 2。此时 `ready == num_nodes`, C 设置 `locked[0]=false` 并 `notifyAll()`, 然后 B 和 C 等待结果。

3. Combination:

- A 被唤醒, 检测到锁释放且人齐了。
- A 计算总和: $1(A) + 1(B) + 1(C) = 3$ 。
- A 携带 3 继续向上遍历。

4. Operation:

- A 到达根节点 (假设当前根的值为 10), 执行加法 $10 + 3 = 13$ 。
- A 获得返回值 10 (这是 A 的返回值)。

5. Distribution:

- A 回到节点 *N*, 调用 `distribute(10)`。
- 计算 B 的结果: $10 + 1 = 11$ 。
- 计算 C 的结果: $11 + 1 = 12$ 。
- 状态设为 `RESULT`, 唤醒 B 和 C。

6. Completion:

- A 返回 10。
- B 醒来, 读取 11, 返回。
- C 醒来, 读取 12, 返回。

最终计数器从 10 变为了 13, 三个线程分别得到了 10, 11, 12, 完全正确。

5 分析 (Analysis)

5.1 正确性 (Correctness)

二叉组合树保证了线性一致性 (Linearizability)。通过在节点处的各种锁机制 (`locked[]`) 和状态机转换, 确保了: 1. 主动线程不会在被动线程数据未就绪时离开。2. 被动线程不会在未收到结果时离开。3. 结果的分发严格遵循前缀和逻辑, 保证返回值的唯一性和连续性。

5.2 性能 (Performance)

设 P 为处理器（线程）数量。

- **树高**: 三叉树的高度为 $\lceil \log_3 P \rceil$ ，而二叉树为 $\lceil \log_2 P \rceil$ 。例如，当 $P = 64$ 时，二叉树高 6 层，三叉树仅需 4 层。
- **延迟**: 在无争用情况下，延迟与树高成正比。三叉树减少了路径长度，理论上延迟更低。
- **吞吐量**: 在高争用情况下，每个节点一次能处理 3 个请求，比二叉树的 2 个请求具有更高的并行聚合能力。
- **空间开销**: 节点结构稍微复杂，增加了数组字段，但节点总数减少（约为二叉树的 $2/3$ ）。

综上所述，三叉组合树在保持软件组合技术优势（减少热点争用）的同时，通过增加节点的扇出（Fan-out），进一步优化了大规模并发场景下的延迟和吞吐量表现。